

Tugas Sistem Operasi dan Arsitektur Komputer

Alya Putri Avianti

1. Dalam sistem multiprogramming, starvation dan deadlock dapat muncul akibat manajemen proses yang buruk. Teliti dan jelaskan bagaimana OS modern seperti Linux menangani kemungkinan starvation pada proses dengan prioritas rendah.

Jawaban:

Starvation terjadi ketika proses yang memiliki prioritas rendah akan terus tertunda karena OS akan menjalankan proses yang memiliki prioritas tinggi terlebih dahulu. Namun, OS modern seperti Linux mencegah kemungkinan ini terjadi dengan menggunakan *Completely Fair Scheduler* (CFS) untuk manajemen proses lebih baik. CFS menggunakan konsep *virtual runtime* yaitu metrik yang melacak waktu eksekusi virtual setiap proses berdasarkan prioritasnya. Proses yang sudah lama menunggu dijalankan akan memiliki virtual runtime yang lebih kecil, sehingga konsep ini dapat meningkatkan peluang proses untuk dijalankan. Konsep ini berfungsi mirip dengan konsep dari proses *aging* (meningkatkan prioritas proses secara bertahap jika proses sudah terlalu lama menunggu untuk dijalankan).

2. Jelaskan bagaimana peran sistem operasi sebagai referee, illusionist, dan glue dapat mendukung pengelolaan proses dalam sistem komputer modern. Sertakan contoh nyata untuk setiap peran.

Jawaban:

- *Referee* : OS berperan untuk mengalokasikan sumber daya seperti CPU, memori, dan I/O ke berbagai proses secara adil dan juga memastikan isolasi antar satu proses dengan proses lainnya sehingga mencegah gangguan pada proses. Contoh nyata peran ini yaitu ketika ada 2 atau lebih aplikasi yang sedang dijalankan, OS akan memastikan bahwa antar proses memiliki waktu CPU masing-masing secara adil serta memori yang sedang diakses salah satu program tidak bisa diakses oleh program lain.
- *Illusionist*: OS berperan untuk membuat ilusi bahwa setiap proses masing-masing memiliki CPU dan memorinya sendiri, sehingga OS akan membuat proses seolah-olah berjalan dengan sumber daya yang tak terbatas. Contoh nyata peran ini yaitu setiap aplikasi yang dijalankan akan berpikir memiliki akses ke memori sistem sepenuhnya.
- *Glue*: OS berperan untuk menyediakan layanan bersama yang memungkinkan untuk berbagi sumber daya. Contoh dari peran ini adalah terdapat 2 aplikasi yang dapat menyimpan data ke sistem file yang sama dan menggunakan jaringan yang sama untuk akses internet.

3. Bagaimana OS menciptakan ilusi banyak prosesor (multiprogramming) pada sistem dengan satu CPU fisik? Jelaskan langkah-langkah pergantian konteks yang terlibat.

Jawaban:

OS membuat ilusi *multiprogramming* dengan menjalankan beberapa proses secara bergantian (*time sharing*). Lalu OS juga menggunakan penjadwalan untuk menentukan proses mana

dulu yang mendapatkan CPU. Kedua hal ini dapat dilakukan dengan *context switch*. *Context switch* adalah ketika OS menyimpan status proses saat ini dan mengambil status proses lain. Langkah-langkah dari *context switch* adalah:

1. Pemicu *context switch* ada tiga yaitu syscall, interrupt, dan trap. Syscall adalah ketika proses meminta *system service* seperti exit. Interrupt ketika kejadian eksternal (misalnya I/O device) memaksa OS untuk menghentikan proses yang dijalankan saat ini. Trap adalah ketika kesalahan internal seperti *segmentation fault*.
2. Menyimpan konteks proses saat ini. OS akan menyimpan status proses yang sedang dijalankan termasuk Program Counter (PC) yaitu alamat instruksi berikutnya, isi register CPU, execution flags dan data stack. Selanjutnya konteks ini disimpan di struktur data seperti *Process Control Block* (PCB).
3. Memilih proses berikutnya. Proses penjadwalan akan digunakan oleh OS untuk memilih proses berikutnya dari antrian proses yang sudah siap dijalankan.
4. Mengambil konteks proses baru. Dikarenakan konteks proses baru, sehingga OS akan mengatur ulang PC ke instruksi yang berikutnya akan dijalankan, mengambil nilai register dan execution flags, serta memperbarui stack pointer.
5. Menjalankan proses baru. OS akan memberikan kendali ke proses baru dengan menjalankan instruksi dari PC yang dimuat. Proses akan berjalan hingga terjadi *context switch* lagi.